

Faculty of Computing · Year 3 · Semester 1. 2026
SE3062 – Artificial Intelligence

Practical 8

Constraint Satisfaction Problems (CSP)

Field	Details
Student ID	IT24101848
Student name	Perera H.L.P.S.
Module	SE3062 – Practical 8
Duration	90 – 120 minutes
Problems used	Australia map colouring; 8-Queens
Language / libraries	Python 3, matplotlib, networkx

1. Introduction and Learning Objectives

A Constraint Satisfaction Problem (CSP) is defined by a set of variables, a domain of allowed values for each variable, and a set of constraints restricting which combinations of values may be used together. This practical models binary CSPs from first principles and compares three solving strategies: plain backtracking, backtracking with variable/value ordering heuristics, and backtracking with constraint propagation (MAC).

Objectives

1. Model binary CSPs formally using variables, domains and constraints.
2. Implement backtracking search from first principles.
3. Integrate the MRV, Degree and LCV heuristics.
4. Implement Forward Checking ideas and the AC-3 arc-consistency algorithm.
5. Run an ablation benchmark comparing assignments and backtracks across all configurations.

2. CSP Formulation

2.1 CSP class

The CSP class stores the variables, per-variable domains, the neighbour (constraint-graph) lists and a binary constraint function constraints(var1, val1, var2, val2). It also keeps two diagnostic counters, assignments_count and backtracks_count, which are used in the final comparison. is_consistent() checks a candidate value only against already-assigned neighbours, and is_complete() checks that every variable is assigned.

2.2 Australia map colouring

- **Variables:** WA, NT, SA, Q, NSW, V, T
- **Domains:** {Red, Green, Blue} for every variable
- **Constraints:** adjacent regions must have different colours (val1 != val2). Tasmania (T) has no neighbours.

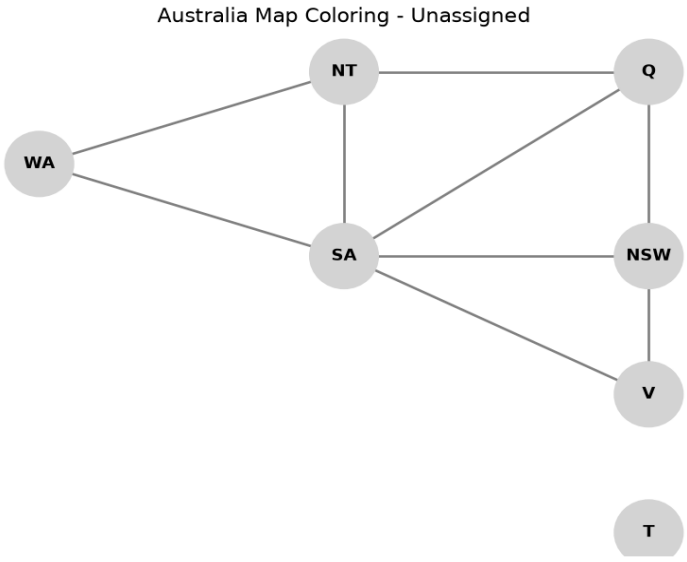


Figure 1: Constraint graph of the Australia map (unassigned)

3. Task 1 – Naive Backtracking Search

Naive backtracking is depth-first search over partial assignments. Because assignments are commutative, only one variable is chosen per level. The steps are: (1) return the assignment if complete; (2) pick the first unassigned variable in static order; (3) for each value, if consistent, assign it, recurse, and on failure undo the assignment and count a backtrack.

Implementation (TODO 1.1 and 1.2)

```
def select_unassigned_variable_naive(csp, assignment):
    for var in csp.variables:
        if var not in assignment:
            return var
    return None

def backtrack_naive(csp, assignment=None):
    if assignment is None:
        assignment = {}

    # TODO 1.1: complete assignment -> return it
    if csp.is_complete(assignment):
        return assignment

    var = select_unassigned_variable_naive(csp, assignment)

    # TODO 1.2: try each value, recurse, undo on failure
    for val in order_domain_values_naive(csp, var, assignment):
        if csp.is_consistent(var, val, assignment):
            assignment[var] = val
            csp.assignments_count += 1
            result = backtrack_naive(csp, assignment)
            if result is not None:
                return result
            del assignment[var]
            csp.backtracks_count += 1

    return None
```

Output

```
Naive Backtracking Completed.
Solution: {'WA':'Red','NT':'Green','SA':'Blue','Q':'Red','NSW':'Green','V':'Red','T':'Red'}
Assignments: 7 | Backtracks: 0
>>> ALL ASSERTIONS PASSED FOR TASK 1.
```

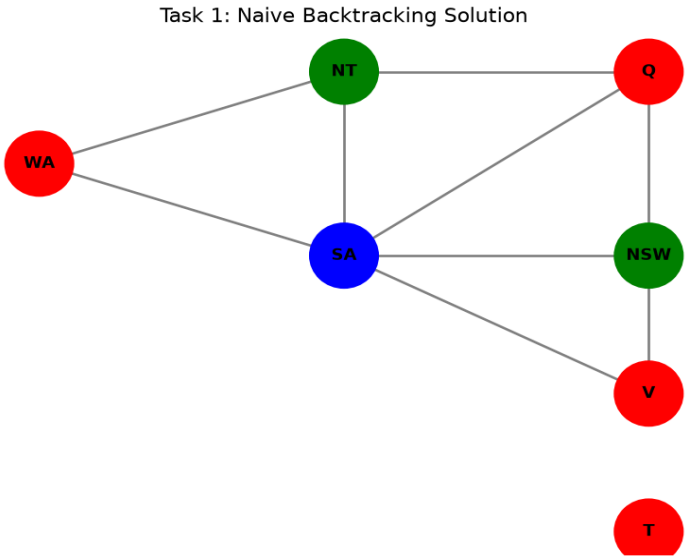


Figure 2: Task 1 – naive backtracking solution

The static order WA, NT, SA, Q, NSW, V, T happens to be favourable for this small map, so a solution is found with 7 assignments (one per variable) and no backtracking. The solution satisfies every adjacency constraint (verified by the assertions).

4. Task 2 – Search Heuristics (MRV, Degree, LCV)

A static order can leave the hardest variables to the end. Two ordering principles are applied:

- **Variable selection – fail-first:** Minimum Remaining Values (MRV) picks the variable with the fewest legal values. Ties are broken by the Degree heuristic: the variable with the most unassigned neighbours.
- **Value selection – succeed-first:** Least Constraining Value (LCV) tries first the value that rules out the fewest options in the domains of unassigned neighbours.

Implementation (TODO 2.1 – 2.4)

```
def select_variable_mrv_degree(csp, assignment):
    unassigned = [v for v in csp.variables if v not in assignment]

    # TODO 2.1: number of legal values left (MRV)
    def legal_count(v):
        return sum(1 for x in csp.domains[v]
                   if csp.is_consistent(v, x, assignment))

    # TODO 2.2: number of unassigned neighbours (Degree)
    def degree(v):
        return sum(1 for n in csp.neighbors[v] if n not in assignment)

    # TODO 2.3: fewest legal values first, ties -> highest degree
    unassigned.sort(key=lambda v: (legal_count(v), -degree(v)))
    return unassigned[0] if unassigned else None

def order_domain_values_lcv(csp, var, assignment):
    # TODO 2.4: count values each choice removes from unassigned neighbours
    def ruled_out(val):
        count = 0
        for n in csp.neighbors[var]:
            if n not in assignment:
                for y in csp.domains[n]:
                    if not csp.constraints(var, val, n, y):
                        count += 1
        return count

    valid = [x for x in csp.domains[var] if csp.is_consistent(var, x, assignment)]
    return sorted(valid, key=ruled_out)
```

Output

```
Heuristic Backtracking Completed.
Solution: {'SA':'Red','NT':'Green','Q':'Blue','NSW':'Green','WA':'Blue','V':'Blue','T':'Red'}
Assignments: 7 | Backtracks: 0
>>> ALL ASSERTIONS PASSED FOR TASK 2.
```

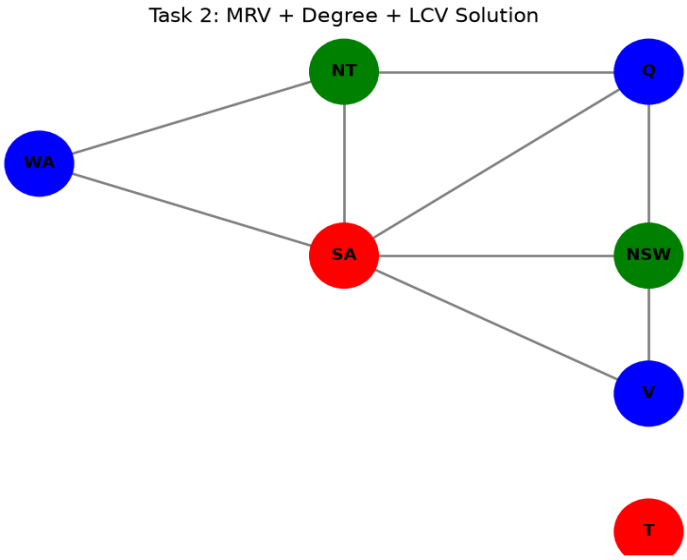


Figure 3: Task 2 – MRV + Degree + LCV solution

On an empty assignment every variable has three legal values, so MRV gives a three-way tie and the Degree heuristic decides. SA has five neighbours, the highest degree, so the first variable chosen is SA – the assertion in the test block confirms this. Choosing the most constrained variable first lets the algorithm solve the map in 7 assignments with 0 backtracks.

5. Task 3 – Forward Checking and AC-3

Constraint propagation prunes domains dynamically so that dead ends are detected before they are explored.

- **Forward Checking:** after assigning var = val, remove conflicting values from unassigned neighbours; fail immediately if any domain becomes empty.
- **AC-3:** keep a queue of directed arcs (Xi, Xj). An arc is revised by deleting every value of Xi that has no supporting value in Xj. If Xi lost a value, arcs (Xk, Xi) for every other neighbour Xk are re-queued; if Xi becomes empty the CSP is inconsistent.

Implementation (TODO 3.1 and 3.2)

```
# Inside the while-queue loop of ac3(csp, queue, domains):
xi, xj = queue.popleft()

# TODO 3.1: revise domains[xi] with respect to xj
revised = False
for x_val in list(domains[xi]):
    if not any(csp.constraints(xi, x_val, xj, y_val) for y_val in domains[xj]):
        domains[xi].remove(x_val)
        revised = True

# TODO 3.2: ripple effect
if revised:
    if not domains[xi]:
        return domains, False          # inevitable failure detected
    for xk in csp.neighbors[xi]:
        if xk != xj:
            queue.append((xk, xi))     # re-check arcs pointing into xi
```

backtrack_mac() combines this with MRV: after each assignment, AC-3 is started with only the arcs (neighbour, var) pointing into the newly assigned variable, using a copy of the domains. If AC-3 reports inconsistency, the value is rejected and a backtrack is counted; otherwise the search continues with the pruned domains (Maintaining Arc Consistency).

Output

```
AC-3 Inconsistency Detection Test:
Consistent: False (Expected: False due to NT-SA collision)
MAC solution: {'WA':'Red','NT':'Green','SA':'Blue','Q':'Red','NSW':'Green','V':'Red','T':'Red'}
Assignments: 7 | Backtracks: 0
>>> ALL ASSERTIONS PASSED FOR TASK 3.
```

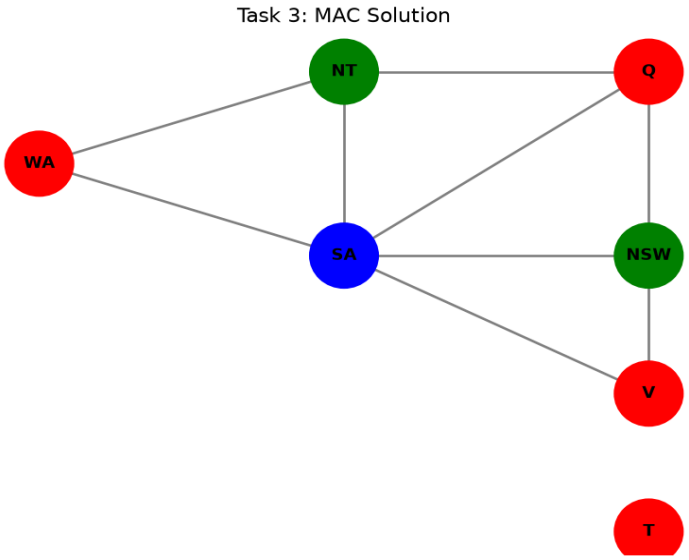


Figure 4: Task 3 – MAC solution

The test domains give NT and SA the same single value (Blue) while NT and SA are neighbours. Revising the arc (NT, SA) deletes Blue from NT, leaving its domain empty, so AC-3 returns False and detects the failure without any search. On the Australia map the MAC search also finds a valid colouring with 7 assignments and no backtracks.

6. Task 4 – Ablation on 8-Queens

The Australia map is too easy to separate the strategies, so all three solvers are benchmarked on 8-Queens. Variable Qi is the row of the queen in column i (domain 0–7). Every pair of variables is constrained by Qi ≠ Qj (same row) and |Qi − Qj| ≠ |i − j| (same diagonal), so the constraint graph is complete.

Results

```
=== Running Ablation Study on 8-Queens Problem ===

Strategy                        | Assignments  | Backtracks   | Time (s)
-------------------------------------------------------------------------
Naive Backtracking              | 113          | 105          | 0.0007
MRV + Degree + LCV              | 89           | 81           | 0.0037
MAC (Maintaining Arc Consistency)  | 20           | 12           | 0.0016
```

Strategy	Assignments	Backtracks	Time (s)
Naive Backtracking	113	105	0.0007
MRV + Degree + LCV	89	81	0.0037
MAC (Maintaining Arc Consistency)	20	12	0.0016

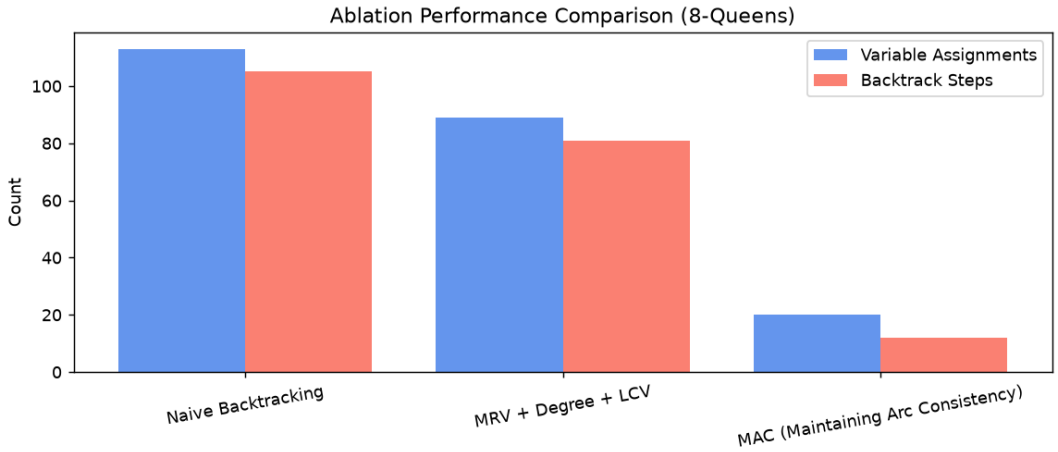


Figure 5: Assignments and backtracks per strategy (8-Queens)

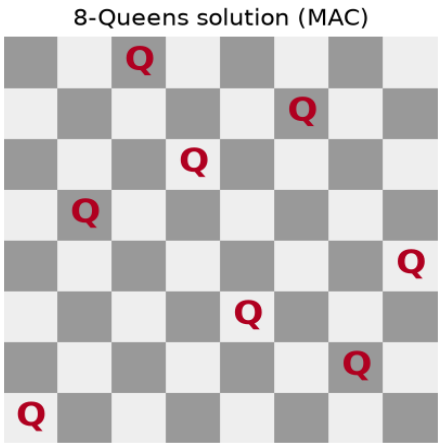


Figure 6: The 8-Queens solution found (columns 0–7 → rows 0, 4, 7, 5, 2, 6, 1, 3)

Timings vary slightly between runs; the assignment and backtrack counts are deterministic.

7. Analysis and Key Observations

1. **Naive backtracking** explored 113 assignments and backtracked 105 times. It only detects a conflict when it reaches the conflicting variable, so early poor choices are discovered late.
2. **MRV + Degree + LCV** reduced this to 89 assignments and 81 backtracks (about 21% fewer assignments). Failing early on the most constrained variable helps, but ordering alone cannot remove values that are already impossible. Its wall-clock time was higher because MRV and LCV do extra work at every node.
3. **MAC** needed only 20 assignments and 12 backtracks (about 82% fewer assignments than naive). Arc consistency prunes impossible values before they are tried, and it is also faster in wall-clock time than the heuristic-only version on this problem.

Trade-off: inference costs more per node (AC-3 and domain copying), so it pays off when it eliminates large parts of the search tree, as it does here. On trivial problems such as the Australia map, all strategies perform similarly.

8. Conclusion

All four tasks were implemented and verified with the supplied assertions. The Australia map was coloured correctly by every strategy, the Degree heuristic selected SA first, and AC-3 detected the NT–SA inconsistency. On 8-Queens, the ablation showed a clear ordering: naive backtracking < heuristics < MAC in efficiency, with MAC reducing the search from 113 to 20 variable assignments. Combining good ordering heuristics with constraint propagation is therefore the most effective way to solve CSPs.